

# Artificial Intelligence

Sem 4

Watermarked study resource

Generated from local SRM dataset

===== SOURCE\_FILE: sem 4/Artificial Intelligence/Notes/Best Short Notes.url =====

CATEGORY: Notes

STATUS: url\_reference\_only

URL: [https://drive.google.com/drive/folders/1n-fPL7UX\\_bg0uUXvZfpaeyJotPEnbZa?usp=drive\\_link](https://drive.google.com/drive/folders/1n-fPL7UX_bg0uUXvZfpaeyJotPEnbZa?usp=drive_link)

===== SOURCE\_FILE: sem 4/Artificial Intelligence/Notes/Unit 1.url =====

CATEGORY: Notes

STATUS: url\_reference\_only

URL: [https://docs.google.com/presentation/d/1ZCRSq-FFGaMC10X3vyhu7SvVjFA9KB22/edit?usp=drive\\_link&oid=104856686103320873965&rtpof=true&sd=true](https://docs.google.com/presentation/d/1ZCRSq-FFGaMC10X3vyhu7SvVjFA9KB22/edit?usp=drive_link&oid=104856686103320873965&rtpof=true&sd=true)

===== SOURCE\_FILE: sem 4/Artificial Intelligence/Notes/Unit 4.pdf =====

CATEGORY: Notes

STATUS: ok

[Page 1]

Unit 4

19/12/23 1

[Page 2]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax , semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 2

[Page 3]

#### Knowledge Representation & Reasoning

- The second most important concept in AI
- If we are going to act rationally in our environment, then we must have some way of describing that environment and drawing inferences from that representation.
- how do we describe what we know about the world ?
- how do we describe it concisely ?
- how do we describe it so that we can get hold of the right piece of knowledge when we need it ?
- how do we generate new pieces of knowledge ?
- how do we deal with uncertain knowledge ?

19/12/23 3

[Page 4]

#### Knowledge

##### Declarative Procedural

- Declarative knowledge deals with factoid questions (what is the capital of India? Etc.)
- Procedural knowledge deals with "How"
- Procedural knowledge can be embedded in declarative knowledge

#### Knowledge Representation & Reasoning

19/12/23 4

[Page 5]

#### Planning

Given a set of goals, construct a sequence of actions that achieves those goals:

- often very large search space
- but most parts of the world are independent of most other parts
- often start with goals and connect them to actions
- no necessary connection between order of planning and order of execution
- what happens if the world changes as we execute the plan and/or our actions don't produce the expected results?

19/12/23 5

[Page 6]

#### Learning

- If a system is going to act truly appropriately, then it must be able to change its actions in the light of experience:
- how do we generate new facts from old ?

- how do we generate new concepts ?
  - how do we learn to distinguish different situations in new environments ?
- 19/12/23 6

[Page 7]

19/12/23 7

What is knowledge representation?

- Knowledge representation and reasoning (KR, KRR) is the part of Artificial intelligence which concerned with AI agents thinking and how thinking contributes to intelligent behavior of agents.
- It is responsible for representing information about the real world so that a computer can understand and can utilize this knowledge to solve the complex real world problems such as diagnosis a medical condition or communicating with humans in natural language.
- It is also a way which describes how we can represent knowledge in artificial intelligence. Knowledge representation is not just storing data into some database, but it also enables an intelligent machine to learn from that knowledge and experiences so that it can behave intelligently like a human.

[Page 8]

19/12/23 8

What to Represent?

Following are the kind of knowledge which needs to be represented in AI systems:

- Object: All the facts about objects in our world domain. E.g., Guitars contains strings, trumpets are brass instruments.
- Events: Events are the actions which occur in our world.
- Performance: It describe behavior which involves knowledge about how to do things.
- Meta-knowledge: It is knowledge about what we know.
- Facts: Facts are the truths about the real world and what we represent.
- Knowledge-Base: The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

[Page 9]

Approaches to knowledge Representation

- Representational adequacy the ability to represent all of the kinds of knowledge that are needed in that domain.
- Inferential Adequacy: - the ability to manipulate the representation structures in such a way as to derive new structures corresponding to new knowledge inferred from ol.
- Inferential Efficiency: - the ability to incorporate into the knowledge structure additional information that can be used to focus the attention of the

inference mechanism in the most promising directions.

- **Acquisitioned Efficiency:** - the ability to acquire new information easily. The simplest case involves direct insertion by a person of new knowledge into the database.

19/12/23 9

[Page 10]

Knowledge Representation Issues

- It becomes clear that particular knowledge representation models allow for more specific more powerful problem solving mechanisms that operate on them.
- Examine specific techniques that can be used for representing & manipulating knowledge within programs.
- Representation & Mapping
- Facts :- truths in some relevant world
- These are the things we want to represent.
- Representations of facts in some chosen formalism.
- Things we are actually manipulating. Structuring these entities is as two levels.
- The knowledge level, at which facts concluding each agents behavior & current goals are described.

Internal Representations Facts

English Representations

English understanding English generation

19/12/23 10

[Page 11]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax , semantics and inferences-Propositional logic-
- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 11

[Page 12]

A KNOWLEDGE-BASED AGENT

12

- A knowledge-based agent includes a knowledge base and an inference system.
- A knowledge base is a set of representations of facts of the world.
- Each individual representation is called a sentence.
- The sentences are expressed in a knowledge representation language.
- The agent operates as follows:
  1. It TELLS the knowledge base what it perceives.
  2. It ASKS the knowledge base what action it should perform.
  3. It performs the chosen action.

19/12/23

[Page 13]

Requirements for a Knowledge-Based Agent

1. "what it already knows" [McCarthy '59]

A knowledge base of beliefs.

2. "it must first be capable of being told" [McCarthy '59]

A way to put new beliefs into the knowledge base.

3. "automatically deduces for itself a sufficiently wide class of immediate consequences" [McCarthy '59]

A reasoning mechanism to derive new beliefs from ones already in the knowledge base.

19/12/23 13

[Page 14]

ARCHITECTURE OF A KNOWLEDGE-BASED

AGENT

14

- Knowledge Level.
  - The most abstract level: describe agent by saying what it knows.
  - Example: A taxi agent might know that the Golden Gate Bridge connects San Francisco with the Marin County.
- Logical Level.
  - The level at which the knowledge is encoded into sentences.
  - Example: `Links(GoldenGateBridge, SanFrancisco, MarinCounty)`.
- Implementation Level.
  - The physical representation of the sentences in the logical level.
  - Example: `'(links goldengatebridge sanfrancisco marincounty)`

19/12/23

[Page 15]

THE WUMPUS WORLD ENVIRONMENT

15

- The Wumpus computer game

- The agent explores a cave consisting of rooms connected by passageways.
  - Lurking somewhere in the cave is the Wumpus, a beast that eats any agent that enters its room.
  - Some rooms contain bottomless pits that trap any agent that wanders into the room.
  - Occasionally, there is a heap of gold in a room.
  - The goal is to collect the gold and exit the world without being eaten
- 19/12/23

[Page 16]

#### A TYPICAL WUMPUS WORLD

16/19/12/23

- The agent always starts in the field [1,1].
- The task of the agent is to find the gold, return to the field [1,1] and climb out of the cave.

[Page 17]

#### AGENT IN A WUMPUS WORLD: PERCEPTS

17

- The agent perceives
- a stench in the square containing the Wumpus and in the adjacent squares (not diagonally)
- a breeze in the squares adjacent to a pit
- a glitter in the square where the gold is
- a bump, if it walks into a wall
- a woeful scream everywhere in the cave, if the wumpus is killed
- The percepts are given as a five-symbol list. If there is a stench and a breeze, but no glitter, no bump, and no scream, the percept is [Stench, Breeze, None, None, None]

19/12/23

[Page 18]

#### WUMPUS WORLD ACTIONS

18

- go forward
- turn right 90 degrees
- turn left 90 degrees
- grab: Pick up an object that is in the same square as the agent
- shoot: Fire an arrow in a straight line in the direction the agent is facing. The arrow continues until it either hits and kills the wumpus or hits the outer wall. The agent has

only one arrow, so only the first Shoot action has any effect

- climb is used to leave the cave. This action is only effective in the start square

- die: This action automatically and irretrievably happens if the agent enters a square with a pit or a live wumpus

19/12/23

[Page 19]

#### ILLUSTRATIVE EXAMPLE: WUMPUS WORLD

- Performance measure

- gold +1000,

- death -1000

(falling into a pit or being eaten by the wumpus)

- -1 per step, -10 for using the arrow

- Environment

- Rooms / squares connected by doors.

- Squares adjacent to wumpus are smelly

- Squares adjacent to pit are breezy

- Glitter iff gold is in the same square

- Shooting kills wumpus if you are facing it

- Shooting uses up the only arrow

- Grabbing picks up gold if in same square

- Releasing drops the gold in same square

- Randomly generated at start of game. Wumpus only senses current room.

- Sensors: Stench, Breeze, Glitter, Bump, Scream [perceptual inputs]

- Actuators: Left turn, Right turn, Forward, Grab, Release, Shoot

19/12/23 19

[Page 20]

#### WUMPUS WORLD CHARACTERIZATION

Fully Observable No - only local perception

Deterministic Yes - outcomes exactly specified

Static Yes - Wumpus and Pits do not move

Discrete Yes

Single-agent? Yes - Wumpus is essentially a “natural feature.”

19/12/23 20

[Page 21]

#### EXPLORING A WUMPUS WORLD

Stench, Breeze, Glitter, Bump, Scream

None, none, none, none, none

The knowledge base of the agent

consists of the rules of the

Wumpus world plus the percept

“nothing” in [1,1]

Boolean percept

feature values:

<0, 0, 0, 0, 0>

19/12/23 21

[Page 22]

Stench, Breeze, Glitter, Bump, Scream

None, none, none, none, none

T=0 The KB of the agent consists of  
the rules of the Wumpus world plus  
the percept “nothing” in [1,1].

By inference, the agent’s knowledge  
base also has the information that  
[2,1] and [1,2] are okay.

Added as propositions.

World “known” to agent  
at time = 0.

EXPLORING A WUMPUS WORLD

19/12/23 22

[Page 23]

Stench, Breeze, Glitter, Bump, Scream

@ T = 1 What follows?

Pit(2,2) or Pit(3,1)

None, none, none, none, none A - agent

V - visited

B - breeze

None, breeze, none, none, none

Where next?

T = 0

EXPLORING A WUMPUS WORLD

19/12/23 23

V A/B P?

P?

T = 1

[Page 24]

EXPLORING A WUMPUS WORLD

19/12/23 24

S

Where is Wumpus? Wumpus cannot be in (1,1) or in (2,2) (Why?) → Wumpus in (1,3)

Not breeze in (1,2) → no pit in (2,2); but we know there is

pit in (2,2) or (3,1) → pit in (3,1)

P?  
P?  
1 2 3 4  
1  
2  
3  
4  
Stench, none, none, none, none  
P  
W  
PS  
T=3  
Stench, Breeze, Glitter, Bump, Scream

[Page 25]

P  
W  
P  
We reasoned about the possible states the Wumpus world can be in,  
given our percepts and our knowledge of the rules of the Wumpus  
world.  
I.e., the content of KB at T=3.  
Essence of logical reasoning:  
Given all we know, Pit\_in\_(3,1) holds.  
("The world cannot be different.")  
What follows is what holds true in all those worlds that satisfy what is  
known at that time T=3 about the particular Wumpus world we are in.  
Models(KB) Models(P\_in\_(3,1))  
Example property: P\_in\_(3,1)  
EXPLORING A WUMPUS WORLD  
19/12/23 25

[Page 26]

NO INDEPENDENT ACCESS TO THE WORLD

26  
• The reasoning agent often gets its knowledge about the facts of the world as a sequence of  
logical sentences and must draw conclusions only from them without independent access to  
the world.  
• Thus it is very important that the agent's reasoning is sound!  
19/12/23

[Page 27]

SUMMARY OF KNOWLEDGE BASED AGENTS

27  
• Intelligent agents need knowledge about the world for making good decisions.

- The knowledge of an agent is stored in a knowledge base in the form of sentences in a knowledge representation language.
  - A knowledge-based agent needs a knowledge base and an inference mechanism. It operates by storing sentences in its knowledge base, inferring new sentences with the inference mechanism, and using them to deduce which actions to take.
  - A representation language is defined by its syntax and semantics, which specify the structure of sentences and how they relate to the facts of the world.
  - The interpretation of a sentence is the fact to which it refers. If this fact is part of the actual world, then the sentence is true.
- 19/12/23

[Page 28]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic-
- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 28

[Page 29]

What is a Logic?

- A language with concrete rules
- No ambiguity in representation (may be other errors!)
- Allows unambiguous communication and processing
- Very unlike natural languages e.g. English
- Many ways to translate between languages
- A statement can be represented in different logics
- And perhaps differently in same logic
- Expressiveness of a logic
- How much can we say in this language?
- Not to be confused with logical reasoning
- Logics are languages, reasoning is a process (may use logic)

2930-03-202119/12/23

[Page 30]

## Syntax and Semantics

- Syntax
  - Rules for constructing legal sentences in the logic
  - Which symbols we can use (English: letters, punctuation)
  - How we are allowed to combine symbols
- Semantics
  - How we interpret (read) sentences in the logic
  - Assigns a meaning to each sentence
  - Example: "All lecturers are seven foot tall"
  - A valid sentence (syntax)
  - And we can understand the meaning (semantics)
  - This sentence happens to be false (there is a counterexample)

19/12/23 30

[Page 31]

## Propositional Logic

- Syntax
  - Propositions, e.g. "it is wet"
  - Connectives: and, or, not, implies, iff (equivalent)
  - Brackets, T (true) and F (false)
- Semantics (Classical AKA Boolean)
  - Define how connectives affect truth
  - "P and Q" is true if and only if P is true and Q is true
  - Use truth tables to work out the truth of statements

19/12/23 31

[Page 32]

## Predicate Logic

- Propositional logic combines atoms
  - An atom contains no propositional connectives
  - Have no structure (today\_is\_wet, john\_likes\_apples)
- Predicates allow us to talk about objects
  - Properties: is\_wet(today)
  - Relations: likes(john, apples)
- True or false
  - In predicate logic each atom is a predicate
  - e.g. first order logic, higher-order logic

19/12/23 32

[Page 33]

## First Order Logic

- More expressive logic than propositional
  - Used in this course (Lecture 6 on representation in FOL)
- Constants are objects: john, apples

- Predicates are properties and relations:
  - likes(john, apples)
- Functions transform objects:
  - likes(john, fruit\_of(apple\_tree))
- Variables represent any object: likes(X, apples)
- Quantifiers qualify values of variables
  - True for all objects (Universal):  $\forall X. \text{likes}(X, \text{apples})$
  - Exists at least one object (Existential):  $\exists X. \text{likes}(X, \text{apples})$

19/12/23 33

[Page 34]

Example: FOL Sentence

- “Every rose has a thorn”
- For all X
- if (X is a rose)
- then there exists Y
- (X has Y) and (Y is a thorn)

19/12/23 34

[Page 35]

Example: FOL Sentence

- “On Mondays and Wednesdays I go to John’s house for dinner”
- Note the change from “and” to “or”
- Translating is problematic

19/12/23 35

[Page 36]

Higher Order Logic

- More expressive than first order
- Functions and predicates are also objects
- Described by predicates: binary(addition)
- Transformed by functions: differentiate(square)
- Can quantify over both
- E.g. define red functions as having zero at 17
- Much harder to reason with

19/12/23 36

[Page 37]

Beyond True and False

- Multi-valued logics
- More than two truth values
- e.g., true, false & unknown
- Fuzzy logic uses probabilities, truth value in  $[0,1]$
- Modal logics
- Modal operators define mode for propositions

- Epistemic logics (belief)

- e.g. 

$p$  (necessarily  $p$ ),  $\Diamond p$  (possibly  $p$ ), ...

- Temporal logics (time)

- e.g. 

$p$  (always  $p$ ),  $\Diamond p$  (eventually  $p$ ), ...

19/12/23 37

[Page 38]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 38

[Page 39]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 39

[Page 40]

40

Propositional logic

- Propositional logic consists of:
- The logical values true and false (T and F)
- Propositions: "Sentences," which
- Are atomic (that is, they must be treated as indivisible units, with

no internal structure), and

- Have a single logical value, either true or false
- Operators, both unary and binary; when applied to logical values, yield logical values
- The usual operators are and, or, not, and implies

19/12/23

[Page 41]

41

Truth tables

- Logic, like arithmetic, has operators, which apply to one, two, or more values (operands)
- A truth table lists the results for each possible arrangement of operands
- Order is important:  $x \text{ op } y$  may or may not give the same result as  $y \text{ op } x$
- The rows in a truth table list all possible sequences of truth values for  $n$  operands, and specify a result for each sequence
- Hence, there are  $2^n$  rows in a truth table for  $n$  operands

19/12/23

[Page 42]

42

Unary operators

X Constant true, (T)

T T

F T

- There are four possible unary operators:

- Only the last of these (negation) is widely used (and has a symbol,  $\neg$ , for the operation

X Constant false, (F)

T F

F F

X Identity, (X)

T T

F F

X Negation,  $\neg X$

T F

F T

19/12/23

[Page 43]

43

Combined tables for unary operators

X Constant T Constant F Identity  $\neg X$

T T F T F

F T F F T

[Page 44]

44

Binary operators

- There are sixteen possible binary operators:
- All these operators have names, but I haven't tried to fit them in
- Only a few of these operators are normally used in logic

X Y

T T T T T T T T T F F F F F F F F

T F T T T T F F F F T T T T F F F F

F T T T F F T T F F T T F F T T F F

F F T F T F T F T F T F T F T F T F

19/12/23

[Page 45]

45

Useful binary operators

- Here are the binary operators that are traditionally used:
- Notice in particular that material implication ( $\Rightarrow$ ) only approximately means the same as the English word "implies"
- All the other operators can be constructed from a combination of these (along with unary not,  $\neg$ )

X

Y

AND

$X \wedge Y$

OR

$X \vee Y$

IMPLIES

$X \Rightarrow Y$

BICONDITIONAL

$X \Leftrightarrow Y$

T T T T T T

T F F T F F

F T F T T F

F F F F T T

19/12/23

[Page 46]

46

Logical expressions

- All logical expressions can be computed with some combination of and ( $\wedge$ ), or ( $\vee$ ), and not ( $\neg$ ) operators
- For example, logical implication can be computed this way:
- Notice that  $\neg X \vee Y$  is equivalent to  $X \Rightarrow Y$

$X \vee Y \rightarrow X \vee \neg X \vee Y \quad X \Rightarrow Y$

T T F T T

T F F F F

F T T T T

F F T T T

19/12/23

[Page 47]

$X \vee Y \rightarrow X \vee \neg X \vee Y \quad X \Rightarrow Y$

$X \Rightarrow Y$

AND

$Y \Rightarrow X$

$X \vee Y \Rightarrow X \vee Y$

$X \Rightarrow Y$

AND

$Y \Rightarrow X$

$X \vee Y \Rightarrow X \vee Y$

$X \Rightarrow Y$

AND

$Y \Rightarrow X$

T T F T T T

T F F F T F

F T T T F F

F F T T T T

19/12/23 47

[Page 48]

48

Another example

•Exclusive or (xor) is true if exactly one of its operands is true

•Notice that  $(\neg X \wedge Y) \vee (X \wedge \neg Y)$  is equivalent to  $X \text{ xor } Y$

$X \vee Y \rightarrow X \vee \neg X \vee Y \quad X \wedge \neg Y \quad (\neg X \wedge Y) \vee (X \wedge \neg Y) \quad X \text{ xor } Y$

T T F F F F F F

T F F T F T T T

F T T F T F T T

F F T T F F F F

19/12/23

[Page 49]

Given a propositional logic, check if it is valid

• $(X \rightarrow Y) \vee (X \vee Y)$

•Valid

$X \vee Y \rightarrow X \vee Y \quad X \vee Y \rightarrow X \vee Y$

or

$X \vee Y$

0 0 1 0 1  
0 1 0 1 1  
1 0 1 1 1  
1 1 1 1 1  
19/12/23 49

[Page 50]

50

World

- A world is a collection of prepositions and logical expressions relating those prepositions

- Example:

- Propositions: JohnLovesMary, MaryIsFemale, MaryIsRich

- Expressions:

$\text{MaryIsFemale} \wedge \text{MaryIsRich} \Rightarrow \text{JohnLovesMary}$

- A proposition “says something” about the world, but since it is atomic (you can’t look inside it to see component parts), propositions tend to be very specialized and inflexible

19/12/23

[Page 51]

51

Models

A model is an assignment of a truth value to each proposition, for example:

- JohnLovesMary: T, MaryIsFemale: T, MaryIsRich: F

- An expression is satisfiable if there is a model for which the expression is true

- For example, the above model satisfies the expression

$\text{MaryIsFemale} \wedge \text{MaryIsRich} \Rightarrow \text{JohnLovesMary}$

- An expression is valid if it is satisfied by every model

- This expression is not valid:

$\text{MaryIsFemale} \wedge \text{MaryIsRich} \Rightarrow \text{JohnLovesMary}$

because it is not satisfied by this model:

JohnLovesMary: F, MaryIsFemale: T, MaryIsRich: T

- But this expression is valid:

$\text{MaryIsFemale} \wedge \text{MaryIsRich} \Rightarrow \text{MaryIsFemale}$

19/12/23

[Page 52]

52

Inference rules in propositional logic

- Here are just a few of the rules you can apply when reasoning in propositional logic:

19/12/23

[Page 53]

53

#### Implication elimination

- A particularly important rule allows you to get rid of the implication operator,  $\Rightarrow$  :

- $X \Rightarrow Y \equiv \neg X \vee Y$

- We will use this later on as a necessary tool for simplifying logical expressions

- The symbol  $\equiv$  means “is logically equivalent to”

19/12/23

[Page 54]

54

#### Conjunction elimination

- Another important rule for simplifying logical expressions allows you to get rid of the conjunction (and) operator,  $\wedge$  :

- This rule simply says that if you have an and operator at the top level of a fact (logical expression), you can break the expression up into two separate facts:

- $\text{MaryIsFemale} \wedge \text{MaryIsRich}$

- becomes:

- $\text{MaryIsFemale}$

- $\text{MaryIsRich}$

19/12/23

[Page 55]

55

#### Inference by computer

- To do inference (reasoning) by computer is basically a search process, taking logical expressions and applying inference rules to them

- Which logical expressions to use?

- Which inference rules to apply?

- Usually you are trying to “prove” some particular statement

- Example:

- $\text{it\_is\_raining} \vee \text{it\_is\_sunny}$

- $\text{it\_is\_sunny} \Rightarrow \text{I\_stay\_dry}$

- $A \Rightarrow B = \text{NOT } A \vee B$

- $\text{NOT}(\text{it\_is\_sunny}) \vee \text{I\_stay\_dry}$

- $\text{it\_is\_rainy} \Rightarrow \text{I\_take\_umbrella}$

- $\text{NOT}(\text{it\_is\_rainy}) \vee \text{I\_take\_umbrella}$

- $\text{I\_take\_umbrella} \Rightarrow \text{I\_stay\_dry}$

- $\text{NOT}(\text{I\_take\_umbrella}) \vee \text{I\_stay\_dry}$

- To prove:  $\text{I\_stay\_dry}$

19/12/23

[Page 56]

## Knowledge and Reasoning

### Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 56

[Page 57]

### Reasoning Patterns

- Inference in propositional logic is NP-complete!
- However, inference in propositional logic shows monotonicity:
- Adding more rules to a knowledge base does not affect earlier inferences

19/12/23 57

[Page 58]

58

### Forward and backward reasoning

- Situation: You have a collection of logical expressions (premises), and you are trying to prove some additional logical expression (the conclusion)
- You can:
- Do forward reasoning: Start applying inference rules to the logical expressions you have, and stop if one of your results is the conclusion you want
- Do backward reasoning: Start from the conclusion you want, and try to choose inference rules that will get you back to the logical expressions you have
- With the tools we have discussed so far, neither is feasible

19/12/23

[Page 59]

59

### Example

- Given:
- $it\_is\_raining \vee it\_is\_sunny$
- $it\_is\_sunny \Rightarrow I\_stay\_dry$

- $it\_is\_raining \Rightarrow I\_take\_umbrella$
- $I\_take\_umbrella \Rightarrow I\_stay\_dry$
- You can conclude:
  - $it\_is\_sunny \vee it\_is\_raining$
  - $I\_take\_umbrella \vee it\_is\_sunny$
  - $\neg I\_stay\_dry \Rightarrow I\_take\_umbrella$
- Etc., etc. ... there are just too many things you can conclude!

19/12/23

[Page 60]

60

Predicate calculus

- Predicate calculus is also known as “First Order Logic” (FOL)
- Predicate calculus includes:
  - All of propositional logic
  - Logical values true, false
  - Variables  $x, y, a, b, \dots$
  - Connectives  $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
  - Constants KingJohn, 2, Villanova, ...
  - Predicates Brother,  $>, \dots$
  - Functions Sqrt, MotherOf, ...
  - Quantifiers  $\forall, \exists$

19/12/23

[Page 61]

61

Constants, functions, and predicates

- A constant represents a “thing”--it has no truth value, and it does not occur “bare” in a logical expression
- Examples: DavidMatuszek, 5, Earth, goodIdea
- Given zero or more arguments, a function produces a constant as its value:
  - Examples: motherOf(DavidMatuszek), add(2, 2), thisPlanet()
- A predicate is like a function, but produces a truth value
- Examples: greatInstructor(DavidMatuszek), isPlanet(Earth), greater(3, add(2, 2))

19/12/23

[Page 62]

62

Universal quantification

- The universal quantifier,  $\forall$ , is read as “for each” or “for every”
- Example:  $\forall x, x^2 \geq 0$  (for all  $x$ ,  $x^2$  is greater than or equal to zero)

•Typically,  $\Rightarrow$  is the main connective with  $\forall$ :

$\forall x, \text{at}(x, \text{Villanova}) \Rightarrow \text{smart}(x)$

means "Everyone at Villanova is smart"

•Common mistake: using  $\wedge$  as the main connective with  $\forall$ :

$\forall x, \text{at}(x, \text{Villanova}) \wedge \text{smart}(x)$

means "Everyone is at Villanova and everyone is smart"

•If there are no values satisfying the condition, the result is true

•Example:  $\forall x, \text{isPersonFromMars}(x) \Rightarrow \text{smart}(x)$  is true

19/12/23

[Page 63]

63

Existential quantification

•The existential quantifier,  $\exists$ , is read "for some" or "there exists"

•Example:  $\exists x, x^2 < 0$  (there exists an  $x$  such that  $x^2$  is less than zero)

•Typically,  $\wedge$  is the main connective with  $\exists$ :

$\exists x, \text{at}(x, \text{Villanova}) \wedge \text{smart}(x)$

means "There is someone who is at Villanova and is smart"

•Common mistake: using  $\Rightarrow$  as the main connective with  $\exists$ :

$\exists x, \text{at}(x, \text{Villanova}) \Rightarrow \text{smart}(x)$

This is true if there is someone at Villanova who is smart...

...but it is also true if there is someone who is not at Villanova

By the rules of material implication, the result of  $F \Rightarrow T$  is  $T$

19/12/23

[Page 64]

64

Properties of quantifiers

•  $\forall x \forall y$  is the same as  $\forall y \forall x$  LOVES (X,Y)

•  $\exists x \exists y$  is the same as  $\exists y \exists x$

•  $\exists x \forall y$  is not the same as  $\forall y \exists x$

•  $\exists x \forall y \text{Loves}(x,y)$

• "There is a person who loves everyone in the world"

• More exactly:  $\exists x \forall y (\text{person}(x) \wedge \text{person}(y) \Rightarrow \text{Loves}(x,y))$

•  $\forall y \exists x \text{Loves}(x,y)$

• "Everyone in the world is loved by at least one person"

• Quantifier duality: each can be expressed using the other

•  $\forall x \text{Likes}(x, \text{IceCream}) \neg \exists x \neg \text{Likes}(x, \text{IceCream})$

•  $\exists x \text{Likes}(x, \text{Broccoli}) \neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

19/12/23

[Page 65]

65

Parentheses

- Parentheses are often used with quantifiers
- Unfortunately, everyone uses them differently, so don't be upset at any usage you see
- Examples:
  - $(\forall x) \text{ person}(x) \Rightarrow \text{likes}(x, \text{iceCream})$
  - $(\forall x) (\text{person}(x) \Rightarrow \text{likes}(x, \text{iceCream}))$
  - $(\forall x) [\text{person}(x) \Rightarrow \text{likes}(x, \text{iceCream})]$
  - $\forall x, \text{ person}(x) \Rightarrow \text{likes}(x, \text{iceCream})$
  - $\forall x (\text{person}(x) \Rightarrow \text{likes}(x, \text{iceCream}))$
- I prefer parentheses that show the scope of the quantifier
- $\exists x (x > 0) \wedge \exists x (x < 0)$

19/12/23

[Page 66]

66

More rules

- Now there are numerous additional rules we can apply!
- Here are two exceptionally important rules:
  - $\neg \forall x, p(x) \Rightarrow \exists x, \neg p(x)$

"If not every  $x$  satisfies  $p(x)$ , then there exists a  $x$  that does not satisfy  $p(x)$ "

- $\neg \exists x, p(x) \Rightarrow \forall x, \neg p(x)$

"If there does not exist an  $x$  that satisfies  $p(x)$ , then all  $x$  do not satisfy  $p(x)$ "

- In any case, the search space is just too large to be feasible
- This was the case until 1970, when J. Robinson discovered resolution

19/12/23

[Page 67]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax , semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 67

[Page 68]

68

Logic by computer was infeasible

- Why is logic so hard?
- You start with a large collection of facts (predicates)
- You start with a large collection of possible transformations (rules)
- Some of these rules apply to a single fact to yield a new fact
- Some of these rules apply to a pair of facts to yield a new fact
- So at every step you must:
- Choose some rule to apply
- Choose one or two facts to which you might be able to apply the rule
- If there are  $n$  facts
- There are  $n$  potential ways to apply a single-operand rule
- There are  $n * (n - 1)$  potential ways to apply a two-operand rule
- Add the new fact to your ever-expanding fact base
- The search space is huge!

19/12/23

[Page 69]

69

The magic of resolution

- Here's how resolution works:
- You transform each of your facts into a particular form, called a clause (this is the tricky part)
- You apply a single rule, the resolution principle, to a pair of clauses
- Clauses are closed with respect to resolution--that is, when you resolve two clauses, you get a new clause
- You add the new clause to your fact base
- So the number of facts you have grows linearly
- You still have to choose a pair of facts to resolve
- You never have to choose a rule, because there's only one

19/12/23

[Page 70]

70

The fact base

- A fact base is a collection of "facts," expressed in predicate calculus, that are presumed to be true (valid)
- These facts are implicitly "anded" together
- Example fact base:
- $\text{seafood}(X) \Rightarrow \text{likes}(\text{John}, X)$  (where  $X$  is a variable)
- $\text{seafood}(\text{shrimp})$
- $\text{pasta}(X) \Rightarrow \neg \text{likes}(\text{Mary}, X)$  (where  $X$  is a different variable)
- $\text{pasta}(\text{spaghetti})$
- That is,
- $(\text{seafood}(X) \Rightarrow \text{likes}(\text{John}, X)) \wedge \text{seafood}(\text{shrimp}) \wedge (\text{pasta}(Y) \Rightarrow \neg \text{likes}(\text{Mary}, Y)) \wedge \text{pasta}(\text{spaghetti})$

- Notice that we had to change some Xs to Ys
- The scope of a variable is the single fact in which it occurs

19/12/23

[Page 71]

71

Clause form

•A clause is a disjunction ("or") of zero or more literals, some or all of which may be negated

•Example:

$\text{sinks}(X) \vee \text{dissolves}(X, \text{water}) \vee \neg \text{denser}(X, \text{water})$

•Notice that clauses use only "or" and "not"—they do not use "and," "implies," or either of the quantifiers "for all" or "there exists"

•The impressive part is that any predicate calculus expression can be put into clause form

•Existential quantifiers,  $\exists$ , are the trickiest ones

19/12/23

[Page 72]

72

Unification

•From the pair of facts (not yet clauses, just facts):

• $\text{seafood}(X) \Rightarrow \text{likes}(\text{John}, X)$  (where X is a variable)

• $\text{seafood}(\text{shrimp})$

•We ought to be able to conclude

• $\text{likes}(\text{John}, \text{shrimp})$

•We can do this by unifying the variable X with the constant shrimp

•This is the same "unification" as is done in Prolog

•This unification turns  $\text{seafood}(X) \Rightarrow \text{likes}(\text{John}, X)$  into

$\text{seafood}(\text{shrimp}) \Rightarrow \text{likes}(\text{John}, \text{shrimp})$

•Together with the given fact  $\text{seafood}(\text{shrimp})$ , the final deductive step is easy

19/12/23

[Page 73]

73

The resolution principle

•Here it is:

•From  $X \vee \text{someLiterals}$

and  $\neg X \vee \text{someOtherLiterals}$

-----

conclude:  $\text{someLiterals} \vee \text{someOtherLiterals}$

•That's all there is to it!

•Example:

• $\text{broke}(\text{Bob}) \vee \text{well-fed}(\text{Bob})$

$\neg \text{broke}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob})$

-----

$\text{well-fed}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob})$

19/12/23

[Page 74]

74

A common error

- You can only do one resolution at a time

- Example:

- $\text{broke}(\text{Bob}) \vee \text{well-fed}(\text{Bob}) \vee \text{happy}(\text{Bob})$

$\neg \text{broke}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob}) \vee \neg \text{happy}(\text{Bob})$

- You can resolve on broke to get:

- $\text{well-fed}(\text{Bob}) \vee \text{happy}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob}) \vee \neg \text{happy}(\text{Bob}) \equiv \text{T}$

- Or you can resolve on happy to get:

- $\text{broke}(\text{Bob}) \vee \text{well-fed}(\text{Bob}) \vee \neg \text{broke}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob}) \equiv \text{T}$

- Note that both legal resolutions yield a tautology (a trivially true statement, containing X

$\vee \neg X$ ), which is correct but useless

- But you cannot resolve on both at once to get:

- $\text{well-fed}(\text{Bob}) \vee \neg \text{hungry}(\text{Bob})$

19/12/23

[Page 75]

75

Contradiction

- A special case occurs when the result of a resolution (the resolvent) is empty, or "NIL"

- Example:

- $\text{hungry}(\text{Bob})$

$\neg \text{hungry}(\text{Bob})$

-----

NIL

- In this case, the fact base is inconsistent

- This will turn out to be a very useful observation in doing resolution theorem proving

19/12/23

[Page 76]

76

A first example

- "Everywhere that John goes, Rover goes. John is at school."

- $\text{at}(\text{John}, X) \Rightarrow \text{at}(\text{Rover}, X)$  (not yet in clause form)

- $\text{at}(\text{John}, \text{school})$  (already in clause form)

- We use implication elimination to change the first of these into clause

form:

- $\neg \text{at}(\text{John}, X) \vee \text{at}(\text{Rover}, X)$
- $\text{at}(\text{John}, \text{school})$
- We can resolve these on  $\text{at}(-, -)$ , but to do so we have to unify  $X$  with  $\text{school}$ ; this gives:
- $\text{at}(\text{Rover}, \text{school})$

19/12/23

[Page 77]

77

Refutation resolution

- The previous example was easy because it had very few clauses
- When we have a lot of clauses, we want to focus our search on the thing we would like to prove
- We can do this as follows:
- Assume that our fact base is consistent (we can't derive NIL)
- Add the negation of the thing we want to prove to the fact base
- Show that the fact base is now inconsistent
- Conclude the thing we want to prove

19/12/23

[Page 78]

78

Example of refutation resolution

- "Everywhere that John goes, Rover goes. John is at school. Prove that Rover is at school."
1.  $\neg \text{at}(\text{John}, X) \vee \text{at}(\text{Rover}, X)$
  2.  $\text{at}(\text{John}, \text{school})$
  3.  $\neg \text{at}(\text{Rover}, \text{school})$  (this is the added clause)
    - Resolve #1 and #3:
  4.  $\neg \text{at}(\text{John}, X)$ 
    - Resolve #2 and #4:
  4. NIL
    - Conclude the negation of the added clause:  $\text{at}(\text{Rover}, \text{school})$
    - This seems a roundabout approach for such a simple example, but it works well for larger problems

19/12/23

[Page 79]

79

A second example

- Start with:
- $\text{it\_is\_raining} \vee \text{it\_is\_sunny}$
- $\text{it\_is\_sunny} \Rightarrow \text{I\_stay\_dry}$
- $\text{it\_is\_raining} \Rightarrow \text{I\_take\_umbrella}$
- $\text{I\_take\_umbrella} \Rightarrow \text{I\_stay\_dry}$

- Convert to clause form:
  1.  $it\_is\_raining \vee it\_is\_sunny$
  2.  $\neg it\_is\_sunny \vee I\_stay\_dry$
  3.  $\neg it\_is\_raining \vee I\_take\_umbrella$
  4.  $\neg I\_take\_umbrella \vee I\_stay\_dry$

- Prove that I stay dry:

5.  $\neg I\_stay\_dry$

- Proof:

6. (5, 2)  $\neg it\_is\_sunny$

7. (6, 1)  $it\_is\_raining$

8. (5, 4)  $\neg I\_take\_umbrella$

9. (8, 3)  $\neg it\_is\_raining$

10. (9, 7) NIL

▪ Therefore,  $\neg(\neg I\_stay\_dry)$

▪  $I\_stay\_dry$

19/12/23

[Page 80]

80

Converting sentences to CNF

1. Eliminate all  $\leftrightarrow$  connectives

$$(P \leftrightarrow Q) \Rightarrow ((P \rightarrow Q) \wedge (Q \rightarrow P))$$

2. Eliminate all  $\rightarrow$  connectives

$$(P \rightarrow Q) \Rightarrow (\neg P \vee Q)$$

3. Reduce the scope of each negation symbol to a single predicate

$$\neg\neg P \Rightarrow P$$

$$\neg(P \vee Q) \Rightarrow \neg P \wedge \neg Q$$

$$\neg(P \wedge Q) \Rightarrow \neg P \vee \neg Q$$

$$\neg(\forall x)P \Rightarrow (\exists x)\neg P$$

$$\neg(\exists x)P \Rightarrow (\forall x)\neg P$$

4. Standardize variables: rename all variables so that each quantifier has its own unique variable name

19/12/23

[Page 81]

81

Converting sentences to clausal form Skolem

constants and functions

5. Eliminate existential quantification by introducing Skolem

constants/functions

$$(\exists x)P(x) \Rightarrow P(c)$$

$c$  is a Skolem constant (a brand-new constant symbol that is not used in any other sentence)

$$(\forall x)(\exists y)P(x,y) \Rightarrow (\forall x)P(x, f(x))$$

since  $\exists$  is within the scope of a universally quantified variable, use a Skolem

function  $f$  to construct a new value that depends on the universally quantified variable

$f$  must be a brand-new function name not occurring in any other sentence in the KB.

E.g.,  $(\forall x)(\exists y)\text{loves}(x,y) \Rightarrow (\forall x)\text{loves}(x,f(x))$

In this case,  $f(x)$  specifies the person that  $x$  loves

19/12/23

[Page 82]

82

Converting sentences to clausal form

6. Remove universal quantifiers by (1) moving them all to the left end; (2) making the scope of each the entire sentence; and (3) dropping the “prefix” part

Ex:  $(\forall x)P(x) \Rightarrow P(x)$

7. Put into conjunctive normal form (conjunction of disjunctions) using distributive and associative laws

$(P \wedge Q) \vee R \Rightarrow (P \vee R) \wedge (Q \vee R)$

$(P \vee Q) \vee R \Rightarrow (P \vee Q \vee R)$

8. Split conjuncts into separate clauses

9. Standardize variables so each clause contains only variable names that do not occur in any other clause

19/12/23

[Page 83]

83

An example

$(\forall x)(P(x) \rightarrow ((\forall y)(P(y) \rightarrow P(f(x,y))) \wedge \neg(\forall y)(Q(x,y) \rightarrow P(y))))$

2. Eliminate  $\rightarrow$

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge \neg(\forall y)(\neg Q(x,y) \vee P(y))))$

3. Reduce scope of negation

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (\exists y)(Q(x,y) \wedge \neg P(y))))$

4. Standardize variables

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (\exists z)(Q(x,z) \wedge \neg P(z))))$

5. Eliminate existential quantification

$(\forall x)(\neg P(x) \vee ((\forall y)(\neg P(y) \vee P(f(x,y))) \wedge (Q(x,g(x)) \wedge \neg P(g(x)))))$

6. Drop universal quantification symbols

$(\neg P(x) \vee ((\neg P(y) \vee P(f(x,y))) \wedge (Q(x,g(x)) \wedge \neg P(g(x)))))$

19/12/23

[Page 84]

84

Example

7. Convert to conjunction of disjunctions

$(\neg P(x) \vee \neg P(y) \vee P(f(x,y))) \wedge (\neg P(x) \vee Q(x,g(x))) \wedge$   
 $(\neg P(x) \vee \neg P(g(x)))$

#### 8. Create separate clauses

$\neg P(x) \vee \neg P(y) \vee P(f(x,y))$

$\neg P(x) \vee Q(x,g(x))$

$\neg P(x) \vee \neg P(g(x))$

#### 9. Standardize variables

$\neg P(x) \vee \neg P(y) \vee P(f(x,y))$

$\neg P(z) \vee Q(z,g(z))$

$\neg P(w) \vee \neg P(g(w))$

19/12/23

[Page 85]

85

#### Resolution

- Resolution is a sound and complete inference procedure for FOL
- Reminder: Resolution rule for propositional logic:
  - $P_1 \vee P_2 \vee \dots \vee P_n$
  - $\neg P_1 \vee Q_2 \vee \dots \vee Q_m$
  - Resolvent:  $P_2 \vee \dots \vee P_n \vee Q_2 \vee \dots \vee Q_m$
- Examples
  - $P$  and  $\neg P \vee Q$  : derive  $Q$  (Modus Ponens)
  - $(\neg P \vee Q)$  and  $(\neg Q \vee R)$  : derive  $\neg P \vee R$
  - $P$  and  $\neg P$  : derive False [contradiction!]
  - $(P \vee Q)$  and  $(\neg P \vee \neg Q)$  : derive True

19/12/23

[Page 86]

86

#### Resolution in first-order logic

- Given sentences

$P_1 \vee \dots \vee P_n$

$Q_1 \vee \dots \vee Q_m$

- in conjunctive normal form:
- each  $P_i$  and  $Q_i$  is a literal, i.e., a positive or negated predicate symbol with its terms,
- if  $P_j$  and  $\neg Q_k$  unify with substitution list  $\theta$ , then derive the resolvent sentence:  
 $\text{subst}(\theta, P_1 \vee \dots \vee P_{j-1} \vee P_{j+1} \dots P_n \vee Q_1 \vee \dots \vee Q_{k-1} \vee Q_{k+1} \vee \dots \vee Q_m)$
- Example
  - from clause  $P(x, f(a)) \vee P(x, f(y)) \vee Q(y)$
  - and clause  $\neg P(z, f(a)) \vee \neg Q(z)$
  - derive resolvent  $P(z, f(y)) \vee Q(y) \vee \neg Q(z)$
  - using  $\theta = \{x/z\}$

19/12/23

[Page 87]

87

## A resolution proof tree

19/12/23

[Page 88]

88

### Resolution refutation

- Given a consistent set of axioms KB and goal sentence Q, show that  $KB \models Q$
- Proof by contradiction: Add  $\neg Q$  to KB and try to prove false.  
i.e.,  $(KB \models Q) \leftrightarrow (KB \vee \neg Q \models \text{False})$
- Resolution is refutation complete: it can establish that a given sentence Q is entailed by KB, but can't (in general) be used to generate all logical consequences of a set of sentences
- Also, it cannot be used to prove that Q is not entailed by KB.
- Resolution won't always give an answer since entailment is only semidecidable
- And you can't just run two proofs in parallel, one trying to prove Q and the other trying to prove  $\neg Q$ , since KB might not entail either one

19/12/23

[Page 89]

89

### Refutation resolution proof tree

```
-allergies(w) v sneeze(w)  ¬cat(y) v ¬allergic-to-cats(z) v allergies(z)
¬cat(y) v sneeze(z) v ¬allergic-to-cats(z) cat(Felix)
sneeze(z) v ¬allergic-to-cats(z) allergic-to-cats(Lise)
false
¬sneeze(Lise)sneeze(Lise)
w/z
y/Felix
z/Lise
negated query
```

19/12/23

[Page 90]

90

We need answers to the following questions

- How to convert FOL sentences to conjunctive normal form (a.k.a. CNF, clause form): normalization and skolemization
- How to unify two argument lists, i.e., how to find their most general unifier (mgu)  $\theta$ : unification
- How to determine which two clauses in KB should be resolved next (among all resolvable pairs of clauses) : resolution (search) strategy

19/12/23

[Page 91]

91

## Unification

- Unification is a “pattern-matching” procedure
- Takes two atomic sentences, called literals, as input
- Returns “Failure” if they do not match and a substitution list,  $\theta$ , if they do
- That is,  $\text{unify}(p, q) = \theta$  means  $\text{subst}(\theta, p) = \text{subst}(\theta, q)$  for two atomic sentences,  $p$  and  $q$
- $\theta$  is called the most general unifier (mgu)
- All variables in the given two literals are implicitly universally quantified
- To make literals match, replace (universally quantified) variables by terms

19/12/23

[Page 92]

92

### Unification algorithm

procedure  $\text{unify}(p, q, \theta)$

Scan  $p$  and  $q$  left-to-right and find the first corresponding terms where  $p$  and  $q$  “disagree” (i.e.,  $p$  and  $q$  not equal)

If there is no disagreement, return  $\theta$  (success!)

Let  $r$  and  $s$  be the terms in  $p$  and  $q$ , respectively, where disagreement first occurs

If  $\text{variable}(r)$  then {

Let  $\theta = \text{union}(\theta, \{r/s\})$

Return  $\text{unify}(\text{subst}(\theta, p), \text{subst}(\theta, q), \theta)$

} else if  $\text{variable}(s)$  then {

Let  $\theta = \text{union}(\theta, \{s/r\})$

Return  $\text{unify}(\text{subst}(\theta, p), \text{subst}(\theta, q), \theta)$

} else return “Failure”

end

19/12/23

[Page 93]

93

### Unification: Remarks

- Unify is a linear-time algorithm that returns the most general unifier (mgu), i.e., the shortest-length substitution list that makes the two literals match.
- In general, there is not a unique minimum-length substitution list, but unify returns one of minimum length
- A variable can never be replaced by a term containing that variable

Example:  $x/f(x)$  is illegal.

- This “occurs check” should be done in the above pseudo-code before making the recursive calls

19/12/23

[Page 94]

## Unification examples

## •Example:

•parents(x, father(x), mother(Bill))

•parents(Bill, father(Bill), y)

•{x/Bill, y/mother(Bill)}

## •Example:

•parents(x, father(x), mother(Bill))

•parents(Bill, father(y), z)

•{x/Bill, y/Bill, z/mother(Bill)}

## •Example:

•parents(x, father(x), mother(Jane))

•parents(Bill, father(y), mother(y))

## •Failure

19/12/23

[Page 95]

Practice example : Did Curiosity kill the cat

• Jack owns a dog. Every dog owner is an animal lover. No animal lover kills an animal. Either Jack or Curiosity killed the cat, who is named Tuna. Did Curiosity kill the cat?

• These can be represented as follows:

A.  $(\exists x) \text{ Dog}(x) \wedge \text{Owns}(\text{Jack}, x)$

B.  $(\forall x) ((\exists y) \text{ Dog}(y) \wedge \text{Owns}(x, y)) \rightarrow \text{AnimalLover}(x)$

C.  $(\forall x) \text{ AnimalLover}(x) \rightarrow ((\forall y) \text{ Animal}(y) \rightarrow \neg \text{Kills}(x, y))$

D.  $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$

E.  $\text{Cat}(\text{Tuna})$

F.  $(\forall x) \text{ Cat}(x) \rightarrow \text{Animal}(x)$

G.  $\text{Kills}(\text{Curiosity}, \text{Tuna})$  GOAL

Resolution example

19/12/23

[Page 96]

## •Convert to clause form

A1.  $(\text{Dog}(D))$

A2.  $(\text{Owns}(\text{Jack}, D))$

B.  $(\neg \text{Dog}(y), \neg \text{Owns}(x, y), \text{AnimalLover}(x))$

C.  $(\neg \text{AnimalLover}(a), \neg \text{Animal}(b), \neg \text{Kills}(a, b))$

D.  $(\text{Kills}(\text{Jack}, \text{Tuna}), \text{Kills}(\text{Curiosity}, \text{Tuna}))$

E.  $\text{Cat}(\text{Tuna})$

F.  $(\neg \text{Cat}(z), \text{Animal}(z))$

•Add the negation of query:

$\neg G: (\neg \text{Kills}(\text{Curiosity}, \text{Tuna}))$

D is a skolem constant

19/12/23

[Page 97]

97

- The resolution refutation proof

R1:  $\neg G, D, \{\}$  (Kills(Jack, Tuna))

R2: R1, C, {a/Jack, b/Tuna} ( $\sim$ AnimalLover(Jack),  
 $\sim$ Animal(Tuna))

R3: R2, B, {x/Jack} ( $\sim$ Dog(y),  $\sim$ Owns(Jack, y),  
 $\sim$ Animal(Tuna))

R4: R3, A1, {y/D} ( $\sim$ Owns(Jack, D),  
 $\sim$ Animal(Tuna))

R5: R4, A2,  $\{\}$  ( $\sim$ Animal(Tuna))

R6: R5, F, {z/Tuna} ( $\sim$ Cat(Tuna))

R7: R6, E,  $\{\}$  FALSE

19/12/23

[Page 98]

98

- The proof tree

$\neg G$  D

C

B

A1

A2

F

A

R1: K(J,T)

R2:  $\neg AL(J) \vee \neg A(T)$

R3:  $\neg D(y) \vee \neg O(J,y) \vee \neg A(T)$

R4:  $\neg O(J,D), \neg A(T)$

R5:  $\neg A(T)$

R6:  $\neg C(T)$

R7: FALSE

$\{\}$

{a/J, b/T}

{x/J}

{y/D}

$\{\}$

{z/T}

$\{\}$

19/12/23

[Page 99]

## Knowledge and Reasoning

### Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax , semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 99

[Page 100]

### Production Rules

- Condition-Action Pairs
- IF this condition (or premise or antecedent) occurs,  
THEN some action (or result, or conclusion, or  
consequence) will (or should) occur
- IF the traffic light is red AND you have stopped,  
THEN a right turn is OK

19/12/23 100

[Page 101]

### Production Rules

- Each production rule in a knowledge base represents an  
autonomous chunk of expertise
- When combined and fed to the inference engine, the set of rules  
behaves synergistically
- Rules can be viewed as a simulation of the cognitive behaviour  
of human experts
- Rules represent a model of actual human behaviour
- Predominant technique used in expert systems, often in  
conjunction with frames

19/12/23 101

[Page 102]

### Forms of Rules

- IF premise, THEN conclusion
- IF your income is high, THEN your chance of being  
audited by the Inland Revenue is high
- Conclusion, IF premise
- Your chance of being audited is high, IF your income  
is high

19/12/23 102

[Page 103]

#### Forms of Rules

- Inclusion of ELSE
- IF your income is high, OR your deductions are unusual, THEN your chance of being audited is high, OR ELSE your chance of being audited is low
- More complex rules
- IF credit rating is high AND salary is more than £30,000, OR assets are more than £75,000, AND pay history is not "poor," THEN approve a loan up to £10,000, and list the loan in category "B."
- Action part may have more information: THEN "approve the loan" and "refer to an agent"

19/12/23 103

[Page 104]

#### Characteristics of Rules

First Part Second Part

Names Premise

Antecedent

Situation

IF

Conclusion

Consequence

Action

THEN

Nature Conditions, similar to declarative knowledge Resolutions, similar to procedural knowledge

Size Can have many IFs Usually only one conclusion

Statement AND statements All conditions must be true for a conclusion to be true

OR statements If any condition is true, the conclusion is true

104

[Page 105]

#### Rule-based Inference

- Production rules are typically used as part of a production system
- Production systems provide pattern-directed control of the reasoning process
- Production systems have:
  - Productions: set of production rules
  - Working Memory (WM): description of current state of the world

- Recognise-act cycle

19/12/23 105

[Page 106]

Production Systems

Production

Rules

C1→A1

C2→A2

C3→A3

...

Cn→An

Working

Memory

Conflict

Resolution

Conflict

Set

Environment

19/12/23 106

[Page 107]

Recognise-Act Cycle

- Patterns in WM matched against production rule conditions
- Matching (activated) rules form the conflict set
- One of the matching rules is selected (conflict resolution) and fired
- Action of rule is performed
- Contents of WM updated
- Cycle repeats with updated WM

19/12/23 107

[Page 108]

Conflict Resolution

- Reasoning in a production system can be viewed as a type of search
- Selection strategy for rules from the conflict set controls search
- Production system maintains the conflict set as an agenda
- Ordered list of activated rules (those with their conditions satisfied) which have not yet been executed
- Conflict resolution strategy determines where a newly-activated rule is inserted

19/12/23 108

[Page 109]

#### Salience

- Rules may be given a precedence order by assigning a salience value
- Newly activated rules are placed in the agenda above all rules of lower salience, and below all rules with higher salience
- Rule with higher salience are executed first
- Conflict resolution strategy applies between rules of the same salience
- If salience and the conflict resolution strategy can't determine which rule is to be executed next, a rule is chosen at random from the most highly ranked rules

19/12/23 109

[Page 110]

#### Conflict Resolution Strategies

- Depth-first: newly activated rules placed above other rules in the agenda
- Breadth-first: newly activated rules placed below other rules
- Specificity: rules ordered by the number of conditions in the LHS (simple-first or complex-first)
- Least recently fired: fire the rule that was last fired the longest time ago
- Refraction: don't fire a rule unless the WM patterns that match its conditions have been modified
- Recency: rules ordered by the timestamps on the facts that match their conditions

19/12/23 110

[Page 111]

#### Salience

- Salience facilitates the modularization of expert systems in which modules work at different levels of abstraction
- Over-use of salience can complicate a system
- Explicit ordering to rule execution
- Makes behaviour of modified systems less predictable
- Rule of thumb: if two rules have the same salience, are in the same module, and are activated concurrently, then the order in which they are executed should not matter

19/12/23 111

[Page 112]

#### Common Types of Rules

- Knowledge rules, or declarative rules, state all the facts

and relationships about a problem

- Inference rules, or procedural rules, advise on how to solve a problem, given that certain facts are known
- Inference rules contain rules about rules (metarules)
- Knowledge rules are stored in the knowledge base
- Inference rules become part of the inference engine

19/12/23 112

[Page 113]

Major Advantages of Rules

- Easy to understand (natural form of knowledge)
- Easy to derive inference and explanations
- Easy to modify and maintain
- Easy to combine with uncertainty
- Rules are frequently independent

19/12/23 113

[Page 114]

Major Limitations of Rules

- Complex knowledge requires many rules
- Search limitations in systems with many rules

19/12/23 114

[Page 115]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 115

[Page 116]

Semantic Networks

- A semantic network is a structure for representing knowledge as a pattern of interconnected nodes and arcs
- Nodes in the net represent concepts of entities, attributes, events, values

- Arcs in the network represent relationships that hold between the concepts

19/12/23 116

[Page 117]

Semantic Networks

- Semantic networks can show inheritance
- Relationship types - is-a, has-a
- Semantic Nets - visual representation of relationships
- Can be combined with other representation methods

19/12/23 117

[Page 118]

Semantic Networks

Animal

Can breathe

Can eat

Has skin

Bird

Can fly

Has wings

Has feathers

Canary

Can sing

Is yellow

Ostrich

Runs fast

Cannot fly

Is tall

Fish

Can swim

Has fins

Has gills

Salmon

Swims upstream

Is pink

Is edible

is-a

is-a

is-a

is-a

is-a

19/12/23 118

[Page 119]

DOG  
ANIMAL  
HOUND  
BEAGLE  
SNOOPY  
COLLIE  
LASSIE  
SHEEPDOG  
is a  
is a  
is a  
is a  
barks is a  
instance  
instance  
CHARLIE BROWN  
FICTIONAL  
CHARACTER instance  
instance  
instance  
has tail  
moves  
breathes  
size: medium  
size: small  
works sheep  
tracks  
friend of  
Semantic Networks  
19/12/23 119

[Page 120]

Semantic Networks

What does or should a node represent?

- A class of objects?
- An instance of an class?
- The canonical instance of a class?
- The set of all instances of a class?

19/12/23 120

[Page 121]

Semantic Networks

- Semantics of links that define new objects and links that relate existing objects, particularly those dealing with 'intrinsic' characteristics of a given object

- How does one deal with the problems of comparison between objects (or classes of objects) through their attributes?
- Essentially the problem of comparing object instances
- What mechanisms are there are to handle quantification in semantic network formalisms?

19/12/23 121

[Page 122]

Transitive inference, but...

- Clyde is an elephant, an elephant is a mammal: Clyde is a mammal.
- The US President is elected every 4 years, Bush is US President: Bush is elected every 4 years
- My car is a Ford, Ford is a car company: my car is a car company

19/12/23 122

[Page 123]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution
- Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 123

[Page 124]

Frames

- A frame is a knowledge representation formalism based on the idea of a frame of reference.
- A frame is a data structure that includes all the knowledge about a particular object
- Frames organised in a hierarchy Form of object-oriented programming for AI and ES.
- Each frame describes one object
- Special terminology

M. Minsky (1974) A Framework for Representing Knowledge, MIT-AI Laboratory Memo 306

19/12/23 124

[Page 125]

Frames

- There are two types of frame:

- Class Frame
- Individual or Instance Frame
- A frame carries with it a set of slots that can represent objects that are normally associated with a subject of the frame.

19/12/23 125

[Page 126]

Frames

- The slots can then point to other slots or frames. That gives frame systems the ability to carry out inheritance and simple kinds of data manipulation.
- The use of procedures - also called demons in the literature - helps in the incorporation of substantial amounts of procedural knowledge into a particular frame-oriented knowledge base

19/12/23 126

[Page 127]

Frame-based model of semantic  
memory

- Knowledge is organised in a data structure
- Slots in structure are instantiated with particular values for a given instance of data
- ...translation to OO terminology:
- frames == classes or objects
- slots == variables/methods

19/12/23 127

[Page 128]

General Knowledge as Frames

DOG

Fixed

legs: 4

Default

diet: carnivorous

sound: bark

Variable

size:

colour:

COLLIE

Fixed

breed of: DOG

type: sheepdog

Default

size: 65cm  
Variable  
colour:  
19/12/23 128

[Page 129]

MAMMAL:  
subclass: ANIMAL  
has\_part: head  
ELEPHANT  
subclass: MAMMAL  
colour: grey  
size: large  
Nellie  
instance: ELEPHANT  
likes: apples  
19/12/23 129  
General Knowledge as Frames

[Page 130]

Logic underlies Frames  
•  $\forall x \text{ mammal}(x) \Rightarrow \text{has\_part}(x, \text{head})$   
•  $\forall x \text{ elephant}(x) \Rightarrow \text{mammal}(x)$   
•  $\text{elephant}(\text{clyde})$   
∴  
 $\text{mammal}(\text{clyde})$   
 $\text{has\_part}(\text{clyde}, \text{head})$   
19/12/23 130

[Page 131]

MAMMAL:  
subclass: ANIMAL  
has\_part: head  
\*furry: yes  
ELEPHANT  
subclass: MAMMAL  
has\_trunk: yes  
\*colour: grey  
\*size: large  
\*furry: no  
Clyde  
instance: ELEPHANT  
colour: pink  
owner: Fred  
Nellie

instance: ELEPHANT  
size: small  
19/12/23 131  
Logic underlies Frames

[Page 132]

Frames (Contd.)

- Can represent subclass and instance relationships (both sometimes called ISA or “is a”)
- Properties (e.g. colour and size) can be referred to as slots and slot values (e.g. grey, large) as slot fillers
- Objects can inherit all properties of parent class (therefore Nellie is grey and large)
- But can inherit properties which are only typical (usually called default, here starred), and can be overridden
- For example, mammal is typically furry, but this is not so for an elephant

19/12/23 132

[Page 133]

- Provide a concise, structural representation of knowledge in a natural manner
- Frame encompasses complex objects, entire situations or a management problem as a single entity
- Frame knowledge is partitioned into slots
- Slot can describe declarative knowledge or procedural knowledge
- Hierarchy of Frames: Inheritance

19/12/23 133

Frames (Contd.)

[Page 134]

Capabilities of Frames

- Ability to clearly document information about a domain model; for example, a plant's machines and their associated attributes
- Related ability to constrain allowable values of an attribute
- Modularity of information, permitting ease of system expansion and maintenance
- More readable and consistent syntax for referencing domain objects in the rules

19/12/23 134

[Page 135]

Capabilities of Frames

- Platform for building graphic interface with object graphics
- Mechanism to restrict the scope of facts considered during forward or backward chaining
- Access to a mechanism that supports the inheritance of

information down a class hierarchy

- Used as underlying model in standards for accessing KBs (Open Knowledge Base Connectivity - OKBC)

19/12/23 135

[Page 136]

Summary

- Frames have been used in conjunction with other, less well-grounded, representation formalisms, like production systems, when used to build to pre-operational or operational expert systems

- Frames cannot be used efficiently to organise 'a whole computation

19/12/23 136

[Page 137]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 137

[Page 138]

Types of Inference

- Deduction
- Induction
- Abduction

19/12/23 138

[Page 139]

Deduction

- Deriving a conclusion from given axioms and facts
- Also called logical inference or truth preservation

Axiom - All kids are naughty

Fact/Premise - Riya is a kid

Conclusion - Riya is naughty

19/12/23 139

[Page 140]

Induction

- Deriving general rule or axiom from background knowledge and observations

- Riya is a kid
- Riya is naughty

General axiom which is derived is:

Kids are naughty

19/12/23 140

[Page 141]

Abduction

- A premise is derived from a known axiom and some observations
- All kids are naughty
- Riya is naughty

Inference

Riya is a kid

19/12/23 141

[Page 142]

Knowledge and Reasoning

Table of Contents

- Knowledge and reasoning-Approaches and issues of knowledge reasoning-Knowledge base agents
- Logic Basics-Logic-Propositional logic-syntax ,semantics and inferences-Propositional logic- Reasoning patterns
- Unification and Resolution-Knowledge representation using rules-Knowledge representation using semantic nets
- Knowledge representation using frames-Inferences-
- Uncertain Knowledge and reasoning-Methods-Bayesian probability and belief network
- Probabilistic reasoning-Probabilistic reasoning over time
- Other uncertain techniques-Data mining-Fuzzy logic-Dempster -shafer theory

19/12/23 142

[Page 143]

Uncertain knowledge and reasoning

- In real life, it is not always possible to determine the state of the environment as it might not be clear. Due to partially observable or non-deterministic environments, agents may need to handle uncertainty and deal with it.
- Uncertain data: Data that is missing, unreliable, inconsistent or noisy

- Uncertain knowledge: When the available knowledge has multiple causes leading to multiple effects or incomplete knowledge of causality in the domain
- Uncertain knowledge representation: The representations which provides a restricted model of the real system, or has limited expressiveness
- Inference: In case of incomplete or default reasoning methods, conclusions drawn might not be completely accurate. Let's understand this better with the help of an example.
- IF primary infection is bacteria cea
- AND site of infection is sterile
- AND entry point is gastrointestinal tract
- THEN organism is bacteriod (0.7).
- In such uncertain situations, the agent does not guarantee a solution but acts on its own assumptions and probabilities and gives some degree of belief that it will reach the required solution.

19/12/23 143

[Page 144]

•For example, In case of Medical diagnosis consider the rule Toothache = Cavity. This is not complete as not all patients having toothache have cavities. So we can write a more generalized rule Toothache = Cavity V Gum problems V Abscess... To make this rule complete, we will have to list all the possible causes of toothache. But this is not feasible due to the following rules:

- Laziness- It will require a lot of effort to list the complete set of antecedents and consequents to make the rules complete.
- Theoretical ignorance- Medical science does not have complete theory for the domain
- Practical ignorance- It might not be practical that all tests have been or can be conducted for the patients.
- Such uncertain situations can be dealt with using  
Probability theory  
Truth Maintenance systems  
Fuzzy logic.

19/12/23 144

Uncertain knowledge and reasoning

[Page 145]

Probability

- Probability is the degree of likeliness that an event will occur. It provides a certain degree of belief in case of uncertain situations. It is defined over a set of events U and assigns value  $P(e)$  i.e. probability of occurrence of event e in the range  $[0,1]$ . Here each sentence is labeled with a real number in the range of 0 to 1, 0 means the sentence is false and 1 means it is true.
- Conditional Probability or Posterior Probability is the probability of event A given that B has already occurred.
- $P(A|B) = (P(B|A) * P(A)) / P(B)$

- For example,  $P(\text{It will rain tomorrow} | \text{It is raining today})$  represents conditional probability of it raining tomorrow as it is raining today.
- $P(A|B) + P(\text{NOT}(A)|B) = 1$
- Joint probability is the probability of 2 independent events happening simultaneously like rolling two dice or tossing two coins together. For example, Probability of getting 2 on one dice and 6 on the other is equal to  $1/36$ . Joint probability has a wide use in various fields such as physics, astronomy, and comes into play when there are two independent events. The full joint probability distribution specifies the probability of each complete assignment of values to random variables.

19/12/23 145

Uncertain knowledge and reasoning

[Page 146]

Bayes Theorem

• It is based on the principle that every pair of features being classified is independent of each other. It calculates probability  $P(A|B)$  where A is class of possible outcomes and B is given instance which has to be classified.

- $P(A|B) = P(B|A) * P(A) / P(B)$
- $P(A|B)$  = Probability that A is happening, given that B has occurred (posterior probability)
- $P(A)$  = prior probability of class
- $P(B)$  = prior probability of predictor
- $P(B|A)$  = likelihood

19/12/23 146

Uncertain knowledge and reasoning

[Page 147]

Consider the following data. Depending on the weather (sunny, rainy or overcast), the children will play(Y) or not play(N).

Here, the total number of observations = 14

Probability that children will play given that weather is sunny :

$$\begin{aligned}
 P(\text{Yes} | \text{Sunny}) &= P(\text{Sunny} | \text{Yes}) * P(\text{Yes}) / P(\text{Sunny}) \\
 &= 0.33 * 0.64 / 0.36 \\
 &= 0.59
 \end{aligned}$$

19/12/23 147

Uncertain knowledge and reasoning

[Page 148]

It is a probabilistic graphical model for representing uncertain domain and to reason under uncertainty. It consists of nodes representing variables, arcs representing direct connections between

them, called causal correlations. It represents conditional dependencies between random variables through a Directed Acyclic Graph (DAG). A belief network consist of:

1. A DAG with nodes labeled with variable names,
2. Domain for each random variable,
3. Set of conditional probability tables for each variable given its parents, including prior probability for nodes with no parents.

19/12/23 148

Uncertain knowledge and reasoning

[Page 149]

•Let's have a look at the steps followed.

1. Identify nodes which are the random variables and the possible values they can have from the probability domain. The nodes can be boolean (True/ False), have ordered values or integral values.
2. Structure- It is used to represent causal relationships between the variables. Two nodes are connected if one affects or causes the other and the arc points towards the effect. For instance, if it is windy or cloudy, it rains. There is a direct link from Windy/Cloudy to Rains. Similarly, from Rains to Wet grass and Leave, i.e., if it rains, grass will be wet and leave is taken from work.

19/12/23 149

Uncertain knowledge and reasoning

[Page 150]

3. Probability- Quantifying relationship between nodes. Conditional Probability:

• $P(B|A) = P(A|B) * P(B) / P(A)$

•Joint probability:

4. Markov property- Bayesian Belief Networks require assumption of Markov property, i.e., all direct dependencies are shown by using arcs. Here there is no direct connection between it being Cloudy and Taking a leave. But there is one via Rains. Belief Networks which have Markov property are also called independence maps.

19/12/23 150

Uncertain knowledge and reasoning

[Page 151]

## Inference in Belief Networks

- Bayesian Networks provide various types of representations of probability distribution over their variables. They can be conditioned over any subset of their variables, using any direction of reasoning.
- For example, one can perform diagnostic reasoning, i.e. when it Rains, one can update his belief about the grass being wet or if leave is taken from work. In this case reasoning occurs in the opposite direction to the network arcs. Or one can perform predictive reasoning, i.e., reasoning from new information about causes to new beliefs about effects, following direction of the arcs. For example, if the grass is already wet, then the user knows that it has rained and it might have been cloudy or windy. Another form of reasoning involves reasoning about mutual causes of a common effect. This is called inter causal reasoning.
- There are two possible causes of an effect, represented in the form of a 'V'. For example, the common effect 'Rains' can be caused by two reasons 'Windy' and 'Cloudy.' Initially, the two causes are independent of each other but if it rains, it will increase the probability of both the causes. Assume that we know it was windy. This information explains the reasons for the rainfall and lowers probability that it was cloudy.

19/12/23 151

Uncertain knowledge and reasoning

[Page 152]

Thank you

19/12/23 152

===== SOURCE\_FILE: sem 4/Artificial Intelligence/Notes/Unit 5.pdf =====

CATEGORY: Notes

STATUS: ok

[Page 1]

## Unit 5

Planning - planning problem - Simple planning agent - Blocks world problem - Mean Ends analysis Learning - Machine learning - Learning concepts, methods and models

Introduction to expert system – architecture of expert systems.

[Page 2]

Planning

- The task of coming up with a sequence of actions that will achieve a goal is called planning.
- Planning Environments
  1. Classical Planning Environments
    - Fully observable, deterministic, finite, static and discrete.
  2. Non classical Planning Environments
    - Partially observable, stochastic with different algorithms and agent designs.

[Page 3]

Planning Problem

- The planning problem is actually the question how to go to next state or the goal state from the current state. It involves two things 'how' and 'when' .
- The planning problem is defined with:
  1. Domain model
    - The domain model defines the actions along with the objects. It is necessary to specify the operators too that actually describe the action. Along with this, information about actions and state constraints while acting should also be given. This entirely formulates the domain model.
  2. Initial state
    - The initial state is the state where any action is yet to take place (the stage when the exam schedule is put up!).
  3. Goal state (next state)
    - The final state or the goal state is the state which the plan is intended to achieve.

[Page 4]

Basic Planning Representation

- Planning representation involves
  - 1) State Representation
  - 2) Goal Representation
  - 3) Action Representation

[Page 5]

State Representation

- A state is represented with a conjunction of positive literals using
  - Logical Propositions:  $\text{Poor} \wedge \text{Unknown}$
  - FOL literals:  $\text{At}(\text{Plane1}, \text{OMA}) \wedge \text{At}(\text{Plane2}, \text{JFK})$
  - FOL literals must be ground & function-free

- Not allowed:  $\text{At}(x,y)$  or  $\text{At}(\text{Father}(\text{Fred}),\text{Sydney})$
- Closed World Assumption
- What is not stated are assumed false

[Page 6]

#### Goal Representation

- Goal is a partially specified state
- A proposition satisfies a goal if it contains all the atoms of the goal and possibly others.
- Example:  $\text{Rich} \wedge \text{Famous} \wedge \text{Miserable}$  satisfies the goal  $\text{Rich} \wedge \text{Famous}$

[Page 7]

#### Action Representation

[Page 8]

#### Simple planning agent

- A simple planning agent is very similar to problem-solving agents in that it constructs plans that achieve its goals, and then executes them.
- The limitations of the problem-solving approach motivates the design of planning systems.
- To solve a planning problem using a state-space search approach we would let the:
  - initial state = initial situation
  - goal-test predicate = goal state description
  - successor function computed from the set of operators
  - once a goal is found, solution plan is the sequence of operators in the path from the start node to the goal node

[Page 9]

#### Algorithm of a simple planning agent

1. Generate a goal to achieve
  2. Construct a plan to achieve goal from current state
  3. Execute plan until finished
  4. Begin again with new goal
- The agent first generates a goal to achieve, and then constructs a plan to achieve it from the current state.
  - Once it has a plan, it keeps executing it until the plan is finished, then begins again with a new goal.

[Page 10]

#### Languages for Planning Problems

- STRIPS
- Stanford Research Institute Problem Solver

- Historically important
  - ADL
- Action Description Languages
  - PDDL
- Planning Domain Definition Language

[Page 11]

Stanford Research Institute Problem Solver  
(STRIPS)

- Makes use of the first order predicates.
- STRIPS allows function-free literals.
- Example of a robot: The example involves a robot, a cup tea, guest and two rooms. We want the robot to get the tea and give it to the guest.

[Page 12]

Action Description Language (ADL)

- STRIPS language lacks the expressive power.
- It can be extended very well to overcome some of the limitations and ADL does that.
- The properties of ADL are as follows:
  1. It allows negative literals.
  2. It makes use of quantified variables along with the disjunctions and the conjunctions.
  3. Conditional post-conditions are allowed.
  4. Variables with different types at the same time are allowed and also equality property is available.

[Page 13]

Planning Domain Description Language  
(PDDL)

- It is a standardization of the planning languages.
- It is defined by the researchers as a standard language.
- We can say it is a superset of STRIPS and ADL that allows features like
  1. Objects can have type specifications.
  2. It can have negative pre-conditions.
  3. The add and delete lists can be conditional.
  4. In some cases, it also allows numeric values.

[Page 14]

#### Block World

- There are 'N' number of Blocks resting on table with specified sequence.
- Goal is to arrange in desired sequence.
- Available moves
  - Put block on table
  - Put a block on another block top
- State is represented using sequence of blocks in current pos.

[Page 15]

#### Block World

[Page 16]

#### Means - Ends Analysis

- Means-Ends Analysis is problem-solving techniques used in Artificial intelligence for limiting search in AI programs.
- It is a mixture of Backward and forward search technique.
- The means -ends analysis process centers around finding the difference between current state and goal state.
- Search strategies are either reason forward or backward.

#### Steps

1. First, evaluate the difference between Initial State and final State.
2. Select the various operators which can be applied for each difference.
3. Apply the operator at each difference, which reduces the difference between the current state and goal state.

[Page 17]

#### MEA Algorithm

Step 1: Compare CURRENT to GOAL, if there are no differences between both then return Success and Exit.

- Step 2: Else, select the most significant difference and reduce it by doing the following steps until the success or failure occurs.
- A) Select a new operator O which is applicable for the current difference, and if there is no such operator, then signal failure.
- B) Attempt to apply operator O to CURRENT. Make a description of two states.
  - i) O-Start, a state in which O's preconditions are satisfied.
  - ii) O-Result, the state that would result if O were applied in O -start.

- C) If

(First-Part <----- MEA (CURRENT, O-START)

And

(LAST-Part <----- MEA (O-Result, GOAL), are successful, then signal

Success and return the result of combining FIRST-PART, O, and LASTPART.

[Page 18]

Example of Mean-Ends Analysis:

[Page 19]

Example of Mean-Ends Analysis:

[Page 20]

Non-linear planning

- A plan that consists of sub-problems, which are solved simultaneously is said non-linear plan.
- In case of the goal stack planning there are some problems. To achieve any goal, it could have an impact on the one that has been achieved.

There is a concept of constraint posting that comes with non-linear planning. The constraint posting states that the plan can be built by

- 1. Addition of operators or suggesting operators
- 2. Ordering them
- 3. Binding the variables to the operators

[Page 21]

Conditional Planning

- Conditional planning has to work regardless of the outcome of an action.
- The outcome of actions cannot be determined so the environment is said to be nondeterministic.
- It's a way to deal with uncertainty by checking what is actually happening in the environment at predetermined points in the plan. (Conditional Steps)

Example:

- Check whether SFO airport (San Francisco International Airport) is open

Three kind of Environments

- Fully Observable - The agent always knows the current state
- Partially Observable - The agent knows only a certain amount about the actual state. (much more common in real world)
- Unknown - The agent knows nothing about the current state.

[Page 22]

Conditional Planning in Fully Observable Environments

- Agent used conditional steps to check the state of the environment

to decide what to do next.

- Plan information stores in a library

Ex:

- Syntax: If then plan\_A else plan\_B

[Page 23]

#### Reactive Planning

- Reactive planning is planning under uncertainty.
- Makes use of the if-then rules.
- The reactive planners are based on the concept that they should be able to handle an unknown situation too. So, the reaction rules are used that help them in doing so.
- A rule selection is based on the priority and a holding condition that maximises the priority.
- The rule which is at present in execution is said to be active whereas the ones with holding priority (we can call them possible competitors) are pre-active others are inactive.
- A B-tree structure is used in reactive planning, where the things are algorithm selects the rule. Sometimes, no rule can be selected. In such a case, dependent on the algorithm implementation for rule selection.

[Page 24]

#### Machine Learning

- Machine learning is building and exploring of methods for programming computer to make them Learn.

[Page 25]

#### Machine Learning

- Machine Learning Approach • Scope of ML

[Page 26]

#### Machine Learning

##### Goals of machine learning

- To produce learning algorithms with practical value.
- Development and enhancement of computer algorithms and models to meet the decision making requirements in practical scenarios.
- To facilitate in building Intelligent System (IS) that can be used in solving real time problems.

##### Challenges of Machine Learning

- Availability of limited learning data and unknown perspectives.

- Acquiring Accurate , compact and precise knowledge building.
- Require large working memory to store data.
- Focusing Too Much on Algorithms and Theories
- Monitoring and maintenance

[Page 27]

## Machine Learning

Computational structure used in  
Machine learning:

1. Functions.
2. Logic programs and rule sets
3. Finite state machines
4. Grammars
5. Problem solving system

## Types of Machine Learning

- Rote learning
- Learning from observations
- Learning from agents
- Inductive learning
- Decision tree learning
  - Supervised learning
- Unsupervised learning
- Semi-supervised learning
- Ensemble learning
- Discovery based learning
- Learning by problem solving

[Page 28]

- Rote learning is a rudimentary form of learning, which focusses on memorization.
- Here, the values are stored so that these are not re-computed.
- When computations are more expensive than re-computing from scratch, this strategy can save a significant amount of time.
- Selective paging and caching are some of the examples of rote learning.

## ML - Rote Learning

[Page 29]

## ML - Learning from agents

- An agent is defined the computational entity which is capable of receiving the environment and can act based on the situation.
- The agent is composed of learning element performance element and a curiosity element.
- Based on the coordination between these elements the outcome of the agent's behavior is measured.

[Page 30]

#### ML - Inductive Learning

- Inductive learning involves learning generalized rules from specific examples (can think of this as the “inverse” of deduction)
- Main task: given a set of examples, each classified as positive or negative produce a concept description that matches exactly the positive examples.
- The examples are coded in some representation language, e.g. they are coded by a finite set of real-valued features.
- The concept description is in a certain language that is presumably a superset of the language of possible example encodings.
- A “correct” concept description is one that classifies correctly ALL possible examples, not just those given in the training set.

[Page 31]

#### ML - Decision Tree Learning

- The learned function is represented by a decision tree.
- In terms of programming it is also represented as if then rules.
- Decision tree depicts the simple learning from the observation, method.
- Based on the observation, at every node, decision is taken.

[Page 32]

#### ML - Supervised learning

- Uses labelled dataset.
- Supervised learning is the learning algorithm that is provided with the set of training data and the algorithm further induces the classifier to classify the unseen or new data.
- A line (hyperplane) which is generated after learning separating two classes class A and class the in two parts the classifier and the decision-making engine minimize the false positives and false negative.

[Page 33]

#### ML - Unsupervised Learning

- Use unlabeled dataset.
- Learning is more based on similarities and differences which are visible. These differences and similarities are mathematically represented in unsupervised learning
- Grouping and categorization of the objects is based on the understanding of similarities and visualization of their relations

- Unsupervised learning performs hierarchical clustering.

[Page 34]

#### ML - Semi supervised Learning

- Semi supervised learning is developed to cope up with the issues of learning in supervised or unsupervised mode in isolation.
- Semi-supervised learning tries to learn from the labelled as well as unlabeled data.
- Let  $U$  be a set of unlabeled data and  $L$  be a set of labelled data.
- As the learning process the learning approach identifies the unlabeled data  $U$  with reference to a labelled data  $L$  and keeps on labelling the unlabeled data.
- This method is also called as self training in semi supervised learning.

[Page 35]

#### ML - Ensemble Learning

- Ensemble learning method is the one where multiple learners or learning algorithms are trained.
- In ensemble learning method the whole collection or ensemble of hypothesis is selected from the hypothesis space and their predictions are combined.
- In this approach, the learners are referred to as base learners.
- The most commonly used ensemble learning methods are
  1. Boosting:
    - Boosting can probably be defined as the method for generating accurate predictions by combining the rules that are comparatively inaccurate.
    - Boosting works on the weighted training sets. The weights of the training example reflects the importance of training examples.
  2. Bagging:
    - In Bagging, the training data is resampled. This is referred to as bootstrap sampling, where the training data with replacement is taken in the learning approaches.

[Page 36]

#### ML

#### Discovery-based Learning

- In case of discovery learning the learning, takes place without any help of teacher or supervisor.
- It is called unsupervised.
- It can be said to be inquiry-based learning, where the learner on the basis of past experience and knowledge tries to discover the outcomes.

#### Learning by Problem Solving

- In learning from problem solving, various parameters related to solution and problem are considered.
- These parameters are used and effectively desirability of a particular outcome or decision is determined.
- It is typically based on the way a problem is solved and the outcomes at every step.
- Even in pattern-based decision - making, the correct category is determined based on the previous results.

[Page 37]

#### ML - Learning Methods

- Artificial neural network-based learning - Back propagation
- Support vector machines
- Reinforcement learning
- Adaptive learning
- Multi agent based learning
- Distributed learning
- Speedup learning

[Page 38]

#### Artificial neural network based learning

- An Artificial Neural Network (ANN) is a computational model inspired by the human brain's neural structure.
- It consists of interconnected nodes (neurons) organized into layers.
- Information flows through these nodes, and the network adjusts the connection strengths (weights) during training to learn from data, enabling it to recognize patterns, make predictions, and solve various tasks in machine learning and artificial intelligence.

[Page 39]

#### ANN - Back propagation Algorithm

- Back-propagation is the essence of neural net training.
- It is the method of fine-tuning the weights of a neural net based on the error rate obtained in the previous epoch (i.e., iteration).

- Proper tuning of the weights allows you to reduce error rates and to make the model reliable by increasing its generalization.

[Page 40]

#### ANN - Back propagation Algorithm

1. Inputs X, arrive through the preconnected path
2. Input is modeled using real weights W. The weights are usually randomly selected.
3. Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.
4. Calculate the error in the outputs.  
 $\text{Error}_B = \text{Actual Output} - \text{Desired Output}$
5. Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.
6. Keep repeating the process until the desired output is achieved.

[Page 41]

#### Support Vector Machine (SVM)

- Support vector machines (SVM) are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.
- It is a machine learning approach.
- They analyze the large amount of data to identify patterns from them.
- SVMs are based on the idea of finding a hyperplane that best divides a dataset into two classes, as shown in the image below.

[Page 42]

#### Hyperplane

- As a simple example, for a classification task with only two features, hyperplane is a line that linearly separates and classifies a set of data.
- Intuitively, the further from the hyperplane our data points lie, the more confident we

are that they have been correctly classified.

- We therefore want our data points to be as far away from the hyperplane as possible, while still being on the correct side of it.
- So when new testing data are added, whatever side of the hyperplane it lands will decide the class that we assign to it.

[Page 43]

Identify the right hyperplane (Scenario-1):

- Here, we have three hyperplanes (A, B and C). Now, identify the right hyperplane to classify star and circle. Hyperplane “B” has excellently performed this job.

[Page 44]

Identify the right hyperplane (Scenario-2):

- Here, we have three hyperplanes (A, B and C) and all are segregating the classes well. Now, how can we identify the right hyperplane?
  - Here, maximizing the distances between nearest data point (either class) and hyperplane will help us to decide the right hyperplane.
  - This distance is called as Margin. Let’s look at the second fig
  - We can see that the margin for hyperplane C is high as compared to both A and B. Hence, we name the right hyperplane as C.
  - Another lightning reason for selecting the hyperplane with higher margin is robustness. If we select a hyperplane having low margin then there is high chance of misclassification.

[Page 45]

Reinforcement Learning

- Learning from interaction with an environment to achieve some long-term goal that is related to the state of the environment
- The goal is defined by reward signal, which must be maximised.
- Agent must be able to partially/fully sense the environment state and take actions to influence the environment state
- The state is typically described with a feature vector

[Page 46]

## Reinforcement Learning Systems

- Reinforcement learning systems have 4 main elements:

1. Policy-A policy is a mapping from the perceived states of the environment to actions to be taken when in those states

- A reinforcement learning agent uses a policy to select actions given the current environment state
- An on-policy agent learns only about the policy that it is executing
- An off-policy agent learns about a policy or policies different from the one that it is executing.

2. Reward signal

- The reward signal defines the goal
- On each time step, the environment sends a single number called the reward to the reinforcement learning agent,
- The agent's objective is to maximize the total reward that it receives over the long run
- The reward signal is used to alter the policy

[Page 47]

## Reinforcement Learning Systems

3. Value function

- The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting in that state.
- Values must continually be re-estimated from the sequence of observations that an agent makes over its lifetime.

4. Optional model of the environment

- A model of the environment allows inferences to be made about how the environment will behave.
- Example: Given a state and an action to be taken while in that state, the model could predict the next state and the next reward.
- Models are used for planning, which means deciding on a course of action by considering possible future situations before they are experienced

[Page 48]

## Adaptive learning

- Adaptive machine learning algorithms are the machine learning models, where the changes in the environment help in selecting the algorithm or learning method.
- The adaptive learning solves some of the complex problems for which a single learning method is not enough.
- This method is even more appropriate when the environment is continuously changing and real time response is expected.

[Page 49]

## Multi-Agent System

- A system with multiple autonomous entities, with distributed information, computational ability, and possibly divergent interests.
- A multiagent system is one that consists of a number of agents, which interact with one-another
- In the most general case, agents will be acting on behalf of users with different goals and motivations
- To successfully interact, they will require the ability to cooperate, coordinate, and negotiate with each other, much as people do

[Page 50]

#### Distributed learning

- In distributed learning the task of learning is distributed.
- Need for distributed learning - Arises due to large data sets and time constraints.
- More than one agent in different parts of the data set.
- There will be distributed learning algorithms taken part in each partition to get the desired outcome, which would then be combined.
- Multi agent systems can be thought of as a subset of distributed learning.

[Page 51]

#### Speedup learning

- Speedup learning typically deals with speeding up problem solving by effective use of problem solving experience.
- Hence, prior problem solving experience is an input for speedup learning.
- In this learning,
  1. There is no interaction with the environment.
  2. New problems cannot be solved.
- So, speedup learning accelerates the process based on the previous experiences and prior observations.

[Page 52]

#### Expert System

- An expert system is a computer program that is designed to solve complex problems and to provide decision-making ability like a human expert.

- It performs this by extracting knowledge from its knowledge base using the reasoning and inference rules according to the user queries.

[Page 53]

#### Characteristics of Expert System

- High Performance: The expert system provides high performance for solving any type of complex problem of a specific domain with high efficiency and accuracy.
- Understandable: It responds in a way that can be easily understandable by the user. It can take input in human language and provides the output in the same way.
- Reliable: It is much reliable for generating an efficient and accurate output.
- Highly responsive: ES provides the result for any complex query within a very short period of time.

[Page 54]

#### Components of Expert System

[Page 55]

#### Components of Expert System

##### 1. User Interface

- With the help of a user interface, the expert system interacts with the user, takes queries as an input in a readable format, and passes it to the inference engine.
- After getting the response from the inference engine, it displays the output to the user.
- In other words, it is an interface that helps a non-expert user to communicate with the expert system to find a solution.

##### 2. Inference Engine(Rules of Engine)

- The inference engine is known as the brain of the expert system as it is the main processing unit of the system.
- It applies inference rules to the knowledge base to derive a conclusion or deduce new information.
- It helps in deriving an error-free solution of queries asked by the user.

##### 3. Knowledge Base

- The knowledgebase is a type of storage that stores knowledge acquired from the different experts of the particular domain.
- It is considered as big storage of knowledge.
- The more the knowledge base, the more precise will be the Expert System.

[Page 56]

## Architecture of Expert System

[Page 57]

## Architecture of Expert System

### Explanation Facility

- Allows a user to understand how the expert system arrived at certain conclusions or results.
- For example: it allows a doctor to find out the logic or rationale of the diagnosis made by a medical expert system.

### Inference Engine

- Seeks information and relationships from the knowledge base and provides answers, predictions, and suggestions the way a human expert would.

### Knowledge Base

- Stores all relevant information, data, rules, cases, and relationships used by the expert system.
- It uses rules, If-then statements, fuzzy logic

### Knowledge acquisition facility

- Provide convenient and efficient means of capturing and storing all the components of the knowledge base.
- Acts as an interface between experts and the knowledge base.

### User Interface

- Specialized user interface software employed for designing, creating, updating, and using expert systems.
- The main purpose of the user interface is to make the development and use of an expert system easier for users and decision makers

[Page 58]

## Advantages and Limitations of Expert Systems

===== SOURCE\_FILE: sem 4/Artificial Intelligence/PYQs/PYQ CT Papers.url =====

CATEGORY: PYQs

STATUS: url\_reference\_only

URL: [https://drive.google.com/drive/folders/1RrLhgU-P1EN-eu\\_-BN2k0B\\_EqVSHaeOt?usp=drive\\_link](https://drive.google.com/drive/folders/1RrLhgU-P1EN-eu_-BN2k0B_EqVSHaeOt?usp=drive_link)

===== SOURCE\_FILE: sem 4/Artificial Intelligence/PYQs/PYQ Dec 2023.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 2]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 3]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 4]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

===== SOURCE\_FILE: sem 4/Artificial Intelligence/PYQs/PYQ May 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 2]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 3]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 4]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

===== SOURCE\_FILE: sem 4/Artificial Intelligence/PYQs/PYQ Nov 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 2]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 3]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650

[Page 4]

Downloaded by THE HELPER (srmpyqs@gmail.com)

10MoARcPSD|52021650